VAL CoPilot
Architecture & Process Flow

Production architecture for the three-layer monorepo and the cognitive contract-lifecycle procedures introduced in the orchestrator.

Scope: production runtime only

Excluded: USE_OFFLINE_MOCKS, offline cognitive router, test fixtures

Contents

1. Production System Architecture

VAL CoPilot is a three-layer monorepo. The Validation UI submits Bot Framework activities to the Cognitive Routing agent. The agent uses Azure OpenAI tool-calling guided by **SYSTEM_PROMPT**, then invokes MCP tools in the Data Retrieval layer to ground answers in Fabric SQL and Azure AI Search (plus optional PGVector memory).

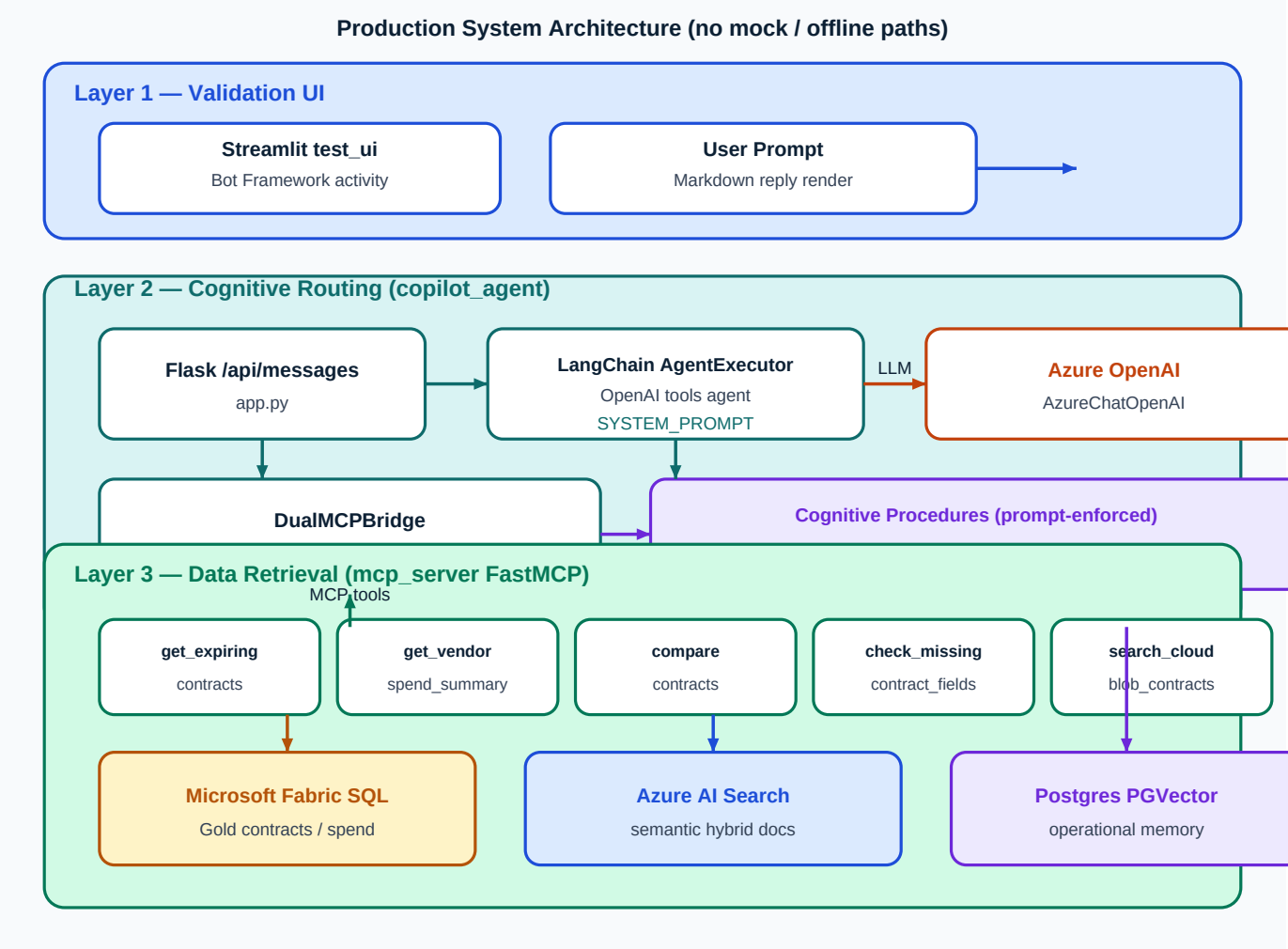

Figure 1 — Layered production architecture

2. End-to-End Request Process Flow

Every production turn follows the same control loop: UI ingress → Flask → LangChain AgentExecutor → Azure OpenAI tool plan → MCP tool execution → evidence synthesis → Markdown reply. No mock interceptors participate in this path.

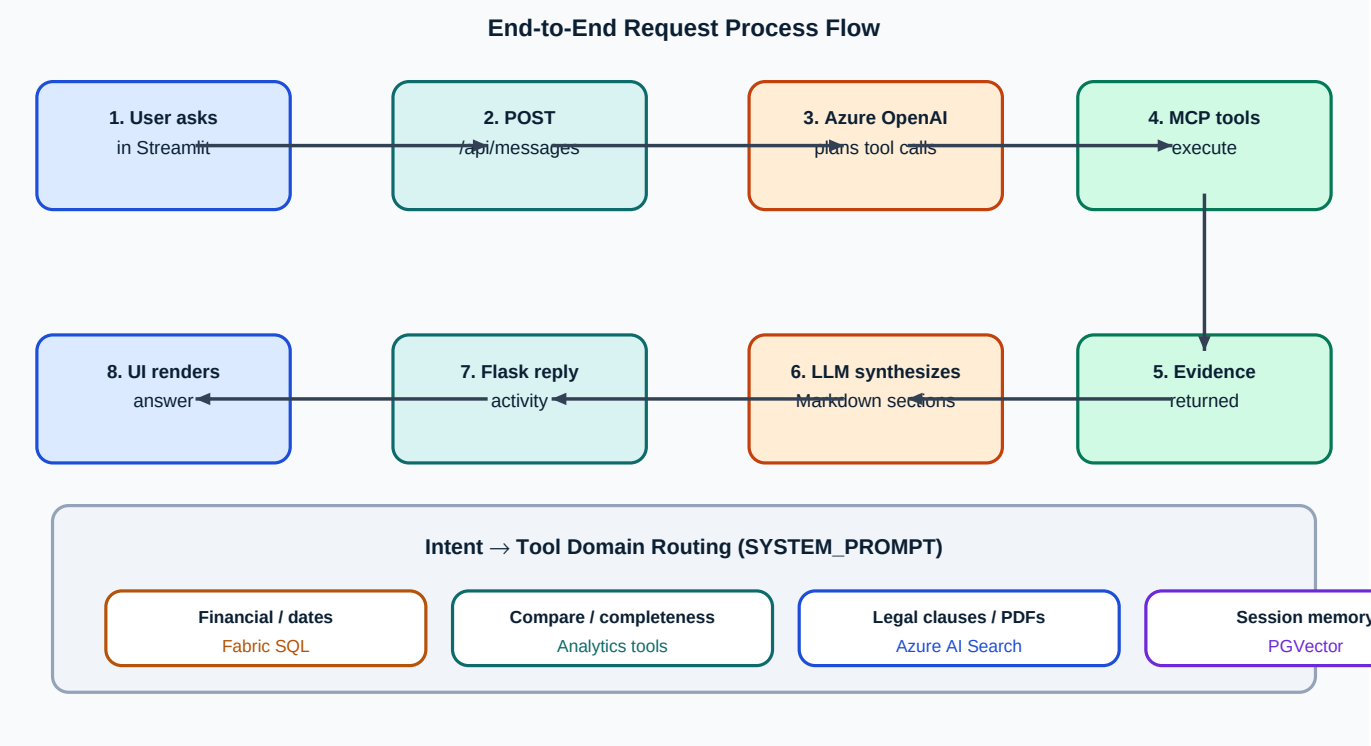

Figure 2 — Request lifecycle

Key production files

- `test_ui/app.py` — Streamlit Bot Framework harness
- `copilot_agent/app.py` — Flask `/api/messages`
- `copilot_agent/agent.py` — SYSTEM_PROMPT + AgentExecutor
- `copilot_agent/mcp_clients.py` — dual MCP stdio bridge
- `mcp_server/server.py` — FastMCP Fabric / Search tools
- `mcp_server/fabric_sql.py` / `azure_search.py` — data access

3. Contract Lifecycle Cognitive Procedures

Advanced analysis is enforced in the orchestrator prompt (not in MCP tool bodies). Each procedure ends in a dedicated Markdown section with actionable recommendations.

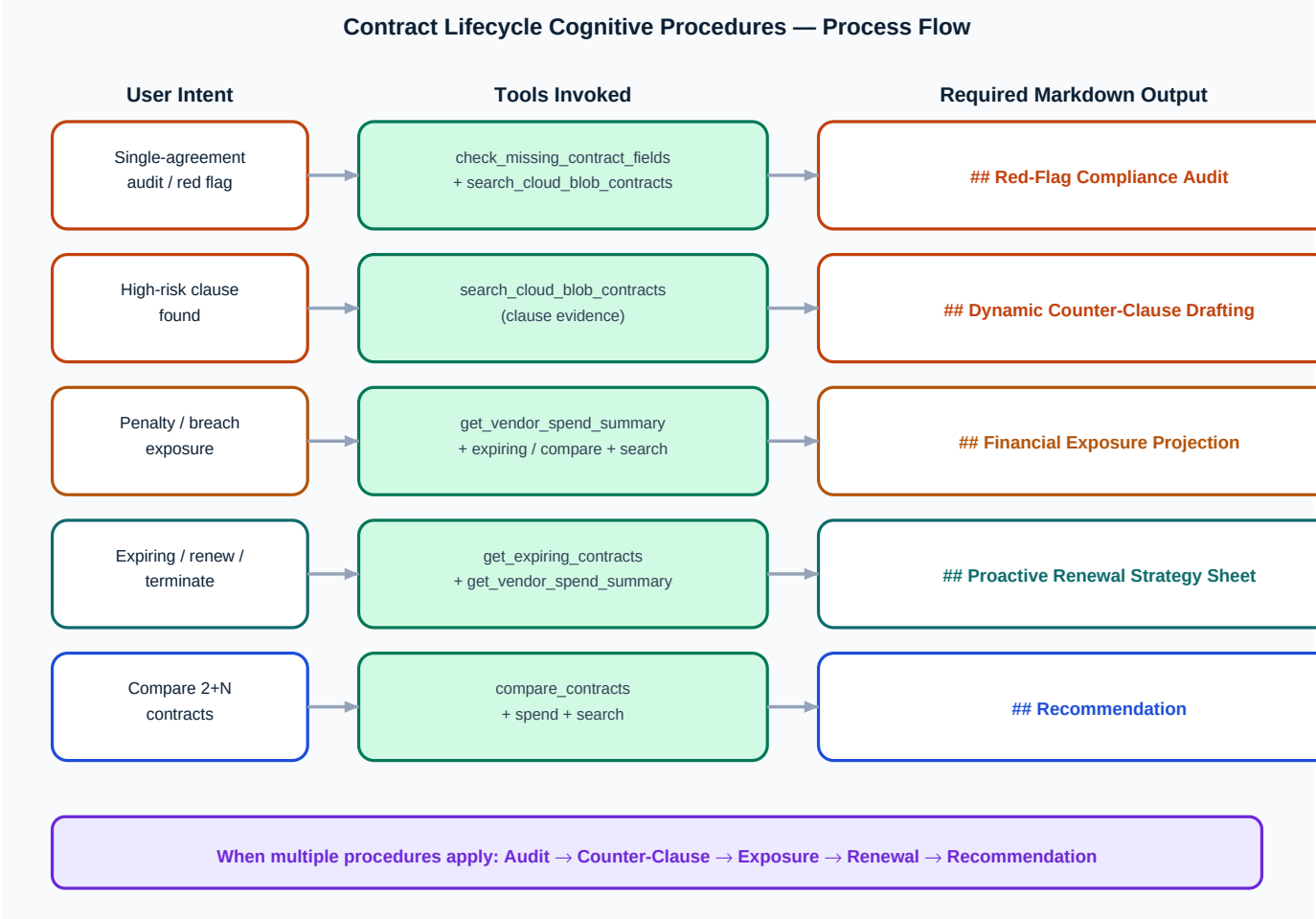

Figure 3 — Intent → tools → Markdown procedure sections

4. Comparative Analysis Decision Framework

For compare intents (including N-way), the agent must not merely juxtapose excerpts. It ranks candidates using quantitative and risk criteria, then emits ## **Recommendation**.

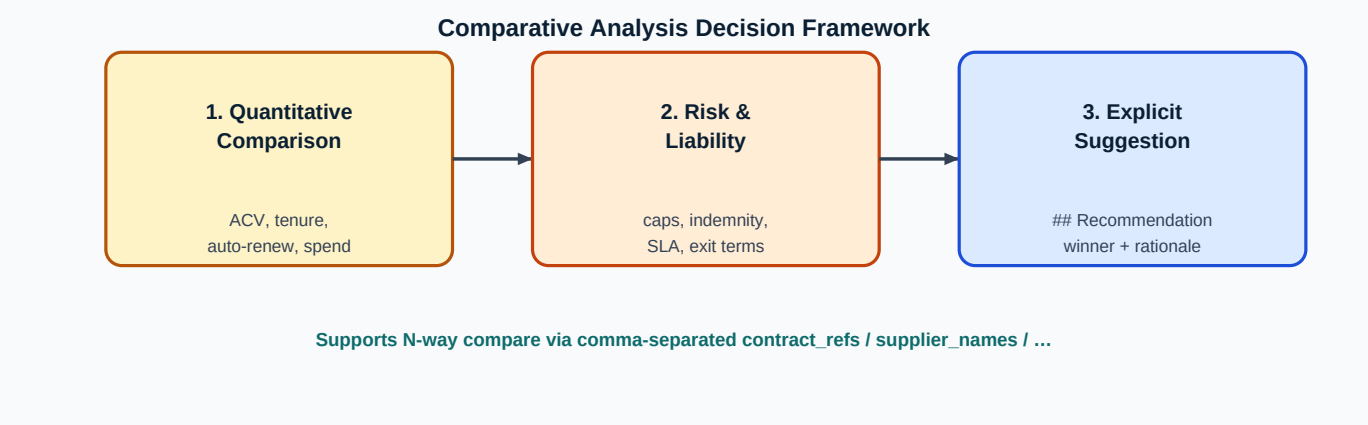

Figure 4 — Compare decision framework

5. Azure AI Foundry Provisioning Path

`copilot_agent/deploy_to_foundry.py` provisions a managed Foundry agent with the same SYSTEM_PROMPT and a FunctionTool ToolSet wrapping core Fabric / Search MCP implementations.

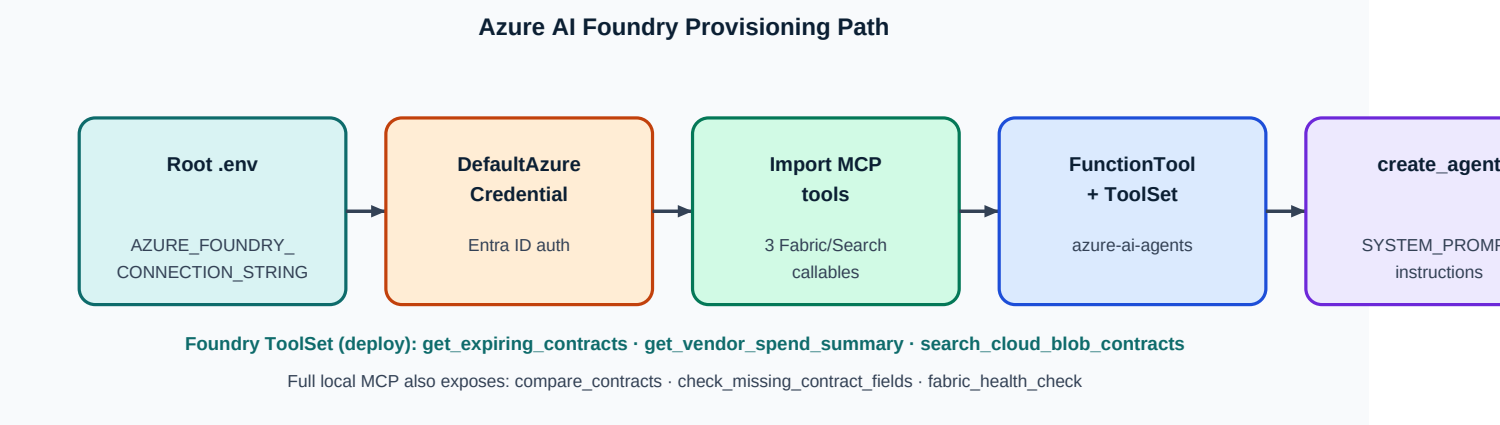

Figure 5 — Foundry deployment process

6. Component & Change Summary

Capabilities delivered in the production architecture (excluding staging mocks):

- **Data Retrieval MCP** — Fabric Gold tools for expiring contracts, vendor spend, N-way compare, missing-field completeness; Azure AI Search hybrid semantic search; shared lookup dimensions (ContractID, SupplierName, ContractName, ContractType, ACV).
- **Cognitive Routing** — LangChain OpenAI-tools agent with strict domain routing; Comparative Analysis framework; four lifecycle procedures with mandatory Markdown sections.
- **Validation UI** — Streamlit emulator posting Bot Framework activities to `POST /api/messages` and rendering Markdown replies.
- **Foundry deploy** — DefaultAzureCredential + ToolSet/FunctionTool provisioning script driven by `AZURE_FOUNDRY_CONNECTION_STRING`.
- **Config boundary** — single root `.env`; MCP remains free of orchestration / LLM logic.

MCP tools (production fabric_data server)

- `get_expiring_contracts`
- `get_vendor_spend_summary`
- `compare_contracts`
- `check_missing_contract_fields`
- `search_cloud_blob_contracts`
- `fabric_health_check`